



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт Информационных Технологий
Кафедра Прикладной математики

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1

по дисциплине
«Технологии и инструментарий анализа больших данных»

Студент группы: ИКБО-04-22

Кликушин В.И.
(Ф. И.О. студента)

Преподаватель

Царев Р.Ю.
(Ф.И.О. преподавателя)

Москва 2025

СОДЕРЖАНИЕ

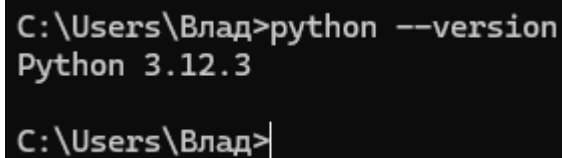
1 Практическая часть	3
1.1 Задание №1	3
1.2 Задание №2	3
1.3 Задание №3	4
1.4 Задание №4	5
1.5 Задание №5	6
1.6 Задание №6	7
1.7 Задание №7	8
1.8 Задание №8	9
1.9 Задание №9	10
1.10 Задание №10	11
1.11 Задание №11	12
1.12 Задание №12	13
1.13 Задание №13	14
1.14 Задание №14	16
1.15 Задание №15	18

1 ПРАКТИЧЕСКАЯ ЧАСТЬ

1.1 Задание №1

Установить Python, если это не было сделано ранее.

Результат установки Python представлен на Рисунке 1.1.1.



```
C:\Users\Влад>python --version
Python 3.12.3
C:\Users\Влад>
```

Рисунок 1.1.1 – Установка Python

1.2 Задание №2

Написать программу, которая вычисляет площадь фигуры, параметры которой подаются на вход. Фигуры, которые подаются на вход: треугольник, прямоугольник, круг. Результатом работы является словарь, где ключ – это название фигуры, а значение – это площадь.

Реализация программы представлена в Листинге А.

Листинг А – Реализация задания №2

```
from math import pi, sqrt

def get_square.figure: str, *args):
    if len(args) == 3:
        p = sum(args) / 2
        a, b, c = args
        return {figure: sqrt(p * (p - a) * (p - b) * (p - c))}
    elif len(args) == 2:
        return {figure: args[0] * args[1]}
    return {figure: pi * args[0] ** 2}

first = "треугольник"
a, b, c = 3, 4, 5
print(get_square(first, a, b, c))

second = "прямоугольник"
a, b = 10, 5
print(get_square(second, a, b))

third = "круг"
r = 10
print(get_square(third, r))
```

Результат тестирования программы представлен на Рисунке 1.2.1.

```
20
21 first = "треугольник"
22 a, b, c = 3, 4, 5
23 print(get_square(first, a, b, c))
24
25 second = "прямоугольник"
26 a, b = 10, 5
27 print(get_square(second, a, b))
28
29 third = "круг"
30 r = 10
31 print(get_square(third, r))
32 |
```

ПРОБЛЕМЫ Выходные данные КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ + 1: Python

- PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/Activate.ps1 (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python
- оги и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.1.py"
- {'треугольник': 6.0}
- {'прямоугольник': 50}
- {'круг': 314.1592653589793}
- o (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>

Рисунок 1.2.1 – Результат тестирования программы для задания №2

1.3 Задание №3

Написать программу, которая на вход получает два числа и операцию, которую к ним нужно применить. Должны быть реализованы следующие операции: +, -, /, //, abs – модуль, pow или ** – возведение в степень.

Реализация программы представлена в Листинге Б.

Листинг Б – Реализация задания №3

```
from operator import mul, add, sub, truediv, floordiv, mod
from math import pow

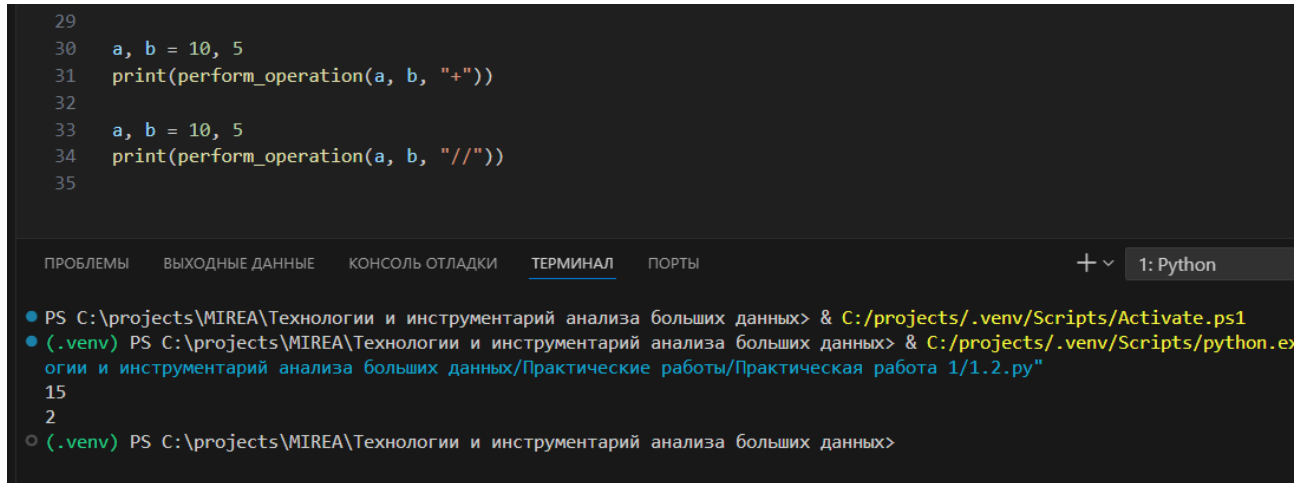
def perform_operation(a, b, operator):
    match operator:
        case "+":
            return add(a, b)
        case "*":
            return mul(a, b)
        case "-":
            return sub(a, b)
        case "/":
            return truediv(a, b)
        case "//":
            return floordiv(a, b)
        case "**":
            return pow(a, b)
        case "abs":
            return mod(a, b)

a, b = 10, 5
print(perform_operation(a, b, "+"))
```

Продолжение Листинга Б

```
a, b = 10, 5
print(perform_operation(a, b, "//"))
```

Результат тестирования программы представлен на Рисунке 1.3.1.



```
29
30 a, b = 10, 5
31 print(perform_operation(a, b, "+"))
32
33 a, b = 10, 5
34 print(perform_operation(a, b, "//"))
35
```

ПРОБЛЕМЫ Выходные данные КОНСОЛЬ ОТЛАДКИ **ТЕРМИНАЛ** ПОРТЫ + 1: Python

- PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/Activate.ps1
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python.exe C:\projects\MIREA\Технологии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.2.py

15
2
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>

Рисунок 1.3.1 – Результат тестирования программы для задания №3

1.4 Задание №4

Напишите программу, которая считывает с консоли числа (по одному в строке) до тех пор, пока сумма введённых чисел не будет равна 0 и после этого выводит сумму квадратов всех считанных чисел.

Реализация программы представлена в Листинге В.

Листинг В – Реализация задания №4

```
n = int(input())
total = n
squares = [n**2]
while total != 0:
    n = int(input())
    total += n
    squares.append(n**2)
print(sum(squares))
```

Результат тестирования программы представлен на Рисунке 1.4.1.

```
0
7  n = int(input())
8  total = n
9  squares = [n**2]
10 while total != 0:
11     n = int(input())
12     total += n
13     squares.append(n**2)
14 print(sum(squares))
15
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

- PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/project
огии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.3.py
5
-5
50
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/project
огии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.3.py
5
7
2
-14
274
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> █

Рисунок 1.4.1 – Результат тестирования программы для задания №4

1.5 Задание №5

Напишите программу, которая выводит последовательность чисел, длиною N, где каждое число повторяется столько раз, чему оно равно. На вход программе передаётся неотрицательное целое число N. Например, если $N = 7$, то программа должна вывести 1 2 2 3 3 3 4. Вывод элементов списка через пробел – `print(*list)`.

Реализация программы представлена в Листинге Г.

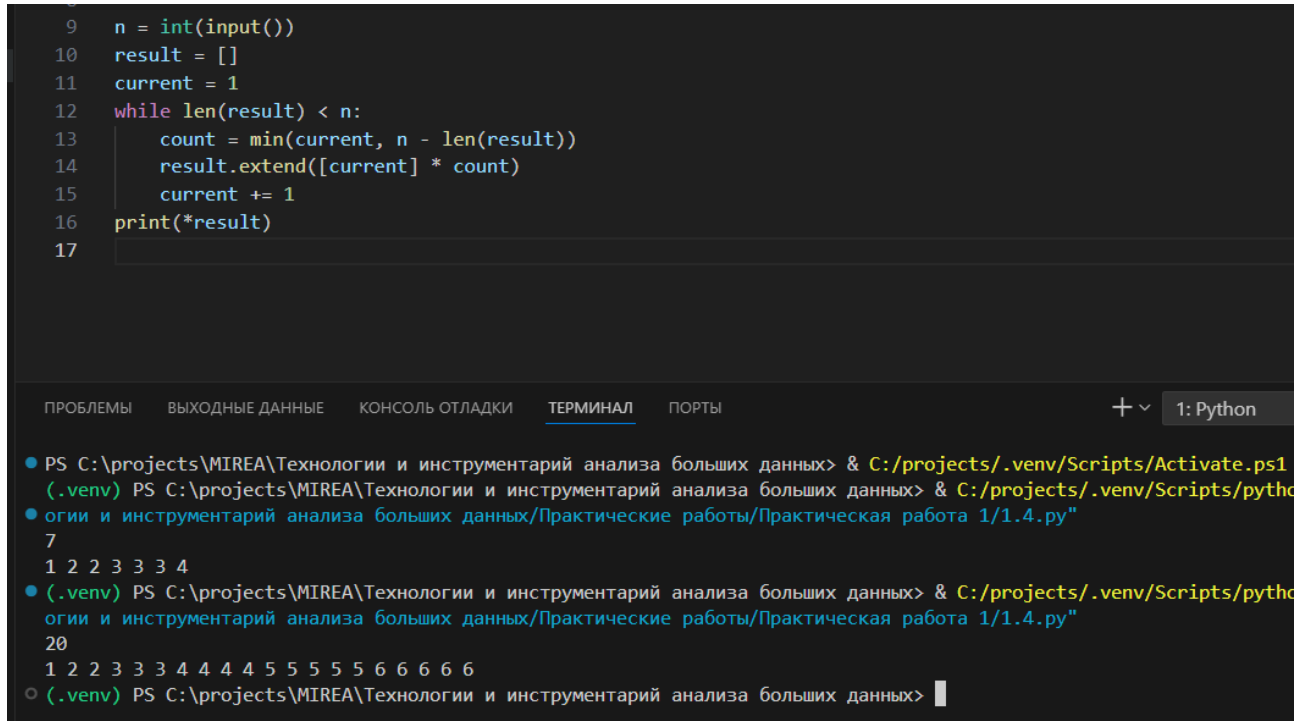
Листинг Г – Реализация задания №5

```
n = int(input())
result = []
current = 1
while len(result) < n:
    count = min(current, n - len(result))
```

Продолжение Листинга Г

```
result.extend([current] * count)
current += 1
print(*result)
```

Результат тестирования программы представлен на Рисунке 1.5.1.



```
9 n = int(input())
10 result = []
11 current = 1
12 while len(result) < n:
13     count = min(current, n - len(result))
14     result.extend([current] * count)
15     current += 1
16 print(*result)
17
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ + 1: Python

- PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/Activate.ps1
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python
- оги и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.4.py"
- 7
- 1 2 2 3 3 3 4
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python
- оги и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.4.py"
- 20
- 1 2 2 3 3 3 4 4 4 4 5 5 5 5 5 6 6 6 6 6
- (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> █

Рисунок 1.5.1 – Результат тестирования программы для задания №5

1.6 Задание №6

Даны два списка: $A = [1, 2, 3, 4, 2, 1, 3, 4, 5, 6, 5, 4, 3, 2]$ $B = ['a', 'b', 'c', 'c', 'c', 'b', 'a', 'c', 'a', 'a', 'b', 'c', 'b', 'a']$ Создать словарь, в котором ключи – это содержимое списка B , а значения для ключей словаря – это сумма всех элементов списка A в соответствии с буквой, содержащийся на той же позиции в списке B . Пример результата программы: $\{'a': 10, 'b': 15, 'c': 6\}$.

Реализация программы представлена в Листинге Д.

Листинг Д – Реализация задания №6

```
A = [1, 2, 3, 4, 2, 1, 3, 4, 5, 6, 5, 4, 3, 2]
B = ['a', 'b', 'c', 'c', 'c', 'b', 'a', 'c', 'a', 'a', 'b', 'c', 'b', 'a']
result = {}
for i in range(len(B)):
    key = B[i]
    value = A[i]
    if key in result:
        result[key] += value
    else:
        result[key] = value
```


Результат тестирования программы представлен на Рисунке 1.7.1.

```
6 from sklearn.datasets import fetch_california_housing
7 import pandas as pd
8
9
10 data = fetch_california_housing(as_frame=True)
11 df = pd.DataFrame(data.data)
12 df["Price"] = data.target
13 print(df.head())
14
15
16 # df.info()
17
18
19 # print(df.isna().sum())
20
21
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```
1 2 2 3 3 3 4
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/
огии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.4.py"
20
1 2 2 3 3 3 4 4 4 4 5 5 5 5 6 6 6 6 6
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/
огии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.5.py"
{'a': 17, 'b': 11, 'c': 17}
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/
огии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.6.py"
   MedInc   HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude  Price
0   8.3252     41.0   6.984127   1.023810      322.0   2.555556     37.88     -122.23   4.526
1   8.3014     21.0   6.238137   0.971880     2401.0   2.109842     37.86     -122.22   3.585
2   7.2574     52.0   8.288136   1.073446      496.0   2.802260     37.85     -122.24   3.521
3   5.6431     52.0   5.817352   1.073059      558.0   2.547945     37.85     -122.25   3.413
4   3.8462     52.0   6.281853   1.081081      565.0   2.181467     37.85     -122.25   3.422
```

Рисунок 1.7.1 – Результат тестирования программы для задания №7

1.8 Задание №8

Использовать метод `info()`.

Реализация программы представлена в Листинге Ж.

Листинг Ж – Реализация задания №8

```
df.info()
```

Результат тестирования программы представлен на Рисунке 1.8.1.

```

6  from sklearn.datasets import fetch_california_housing
7  import pandas as pd
8
9
10 data = fetch_california_housing(as_frame=True)
11 df = pd.DataFrame(data.data)
12 df["Price"] = data.target
13 print(df.head())
14
15
16 df.info()
17
18
19 # print(df.isna().sum())
20
21

```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
8   Price           20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>

```

Рисунок 1.8.1 – Результат тестирования программы для задания №8

1.9 Задание №9

Узнать, есть ли пропущенные значения, используя `isna().sum()`.

Реализация программы представлена в Листинге 3.

Листинг 3 – Реализация задания №9

```
print(df.isna().sum())
```

Результат тестирования программы представлен на Рисунке 1.9.1.

```
1 """
2 Скачать и загрузить данные о стоимости домов в калифорнии,
3 используя библиотеку sklearn.
4 """
5
6 from sklearn.datasets import fetch_california_housing
7 import pandas as pd
8
9
10 data = fetch_california_housing(as_frame=True)
11 df = pd.DataFrame(data.data)
12 df["Price"] = data.target
13 print(df.head())
14
15
16 df.info()
17
18
19 print(df.isna().sum())
20
21
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```
5  AveOccup    20640 non-null float64
6  Latitude    20640 non-null float64
7  Longitude    20640 non-null float64
8  Price        20640 non-null float64
dtypes: float64(9)
memory usage: 1.4 MB
MedInc      0
HouseAge     0
AveRooms     0
AveBedrms    0
Population   0
AveOccup     0
Latitude     0
Longitude     0
Price        0
dtype: int64
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>
```

Рисунок 1.9.1 – Результат тестирования программы для задания №9

1.10 Задание №10

Вывести записи, где средний возраст домов в районе более 50 лет и население более 2500 человек, используя метод loc().

Реализация программы представлена в Листинге И.

Листинг И – Реализация задания №10

```
print(df.loc[(df["HouseAge"] > 50) & (df["Population"] > 2500)])
```

Результат тестирования программы представлен на Рисунке 1.10.1.

```
Практические работы > Практическая работа 1 > 1.6.py > ...
3   используя библиотеку sklearn.
4   """
5
6   from sklearn.datasets import fetch_california_housing
7   import pandas as pd
8
9
10  data = fetch_california_housing(as_frame=True)
11  df = pd.DataFrame(data.data)
12  df["Price"] = data.target
13  print(df.head())
14
15
16  df.info()
17
18
19  print(df.isna().sum())
20
21
22  print(df.loc[(df["HouseAge"] > 50) & (df["Population"] > 2500)])
23
24
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ + ~ 1: Py

Price 0
dtype: int64

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	Price
460	1.4012	52.0	3.105714	1.060000	3337.0	9.534286	37.87	-122.26	1.75000
4131	3.5349	52.0	4.646119	1.047945	2589.0	5.910959	34.13	-118.20	1.93600
4440	2.6806	52.0	4.806283	1.057592	3062.0	4.007853	34.08	-118.21	1.53000
5986	1.8750	52.0	4.500000	1.206349	2688.0	21.333333	34.10	-117.71	2.12500
7369	3.1901	52.0	4.730942	1.017937	3731.0	4.182735	33.97	-118.21	1.67600
8227	2.3305	52.0	3.488860	1.170380	3018.0	3.955439	33.78	-118.20	1.62500
13034	6.1359	52.0	8.275862	1.517241	6675.0	230.172414	38.69	-121.15	2.25000
15634	1.8295	52.0	2.628169	1.053521	2957.0	4.164789	37.80	-122.41	2.43800
15652	0.9000	52.0	2.237474	1.053535	3260.0	2.237474	37.80	-122.41	5.00001
15657	2.5166	52.0	2.839075	1.184049	3436.0	1.621520	37.79	-122.41	2.75000
15659	1.7240	52.0	2.278566	1.082348	4518.0	1.780142	37.79	-122.41	2.25000
15795	2.5755	52.0	3.402576	1.058776	2619.0	2.108696	37.77	-122.42	3.25000
15868	2.8135	52.0	4.584329	1.041169	2987.0	3.966799	37.76	-122.41	2.60300

○ (.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> █

Рисунок 1.10.1 – Результат тестирования программы для задания №10

1.11 Задание №11

Узнать максимальное и минимальное значения медианной стоимости дома.

Реализация программы представлена в Листинге К.

Листинг К – Реализация задания №11

```
print(df["Price"].max())  
print(df["Price"].min())
```

Результат тестирования программы представлен на Рисунке 1.11.1.

```

8
9
10 data = fetch_california_housing(as_frame=True)
11 df = pd.DataFrame(data.data)
12 df["Price"] = data.target
13 print(df.head())
14
15
16 df.info()
17
18
19 print(df.isna().sum())
20
21
22 print(df.loc[(df["HouseAge"] > 50) & (df["Population"] > 2500)])
23
24
25 print(df["Price"].max())
26 print(df["Price"].min())
27
28

```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ **ТЕРМИНАЛ** ПОРТЫ + ▾ 1: Python

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	Price
460	1.4012	52.0	3.105714	1.060000	3337.0	9.534286	37.87	-122.26	1.75000
4131	3.5349	52.0	4.646119	1.047945	2589.0	5.910959	34.13	-118.20	1.93600
4440	2.6806	52.0	4.806283	1.057592	3062.0	4.007853	34.08	-118.21	1.53000
5986	1.8750	52.0	4.500000	1.206349	2688.0	21.333333	34.10	-117.71	2.12500
7369	3.1901	52.0	4.730942	1.017937	3731.0	4.182735	33.97	-118.21	1.67600
8227	2.3305	52.0	3.488860	1.170380	3018.0	3.955439	33.78	-118.20	1.62500
13034	6.1359	52.0	8.275862	1.517241	6675.0	230.172414	38.69	-121.15	2.25000
15634	1.8295	52.0	2.628169	1.053521	2957.0	4.164789	37.80	-122.41	2.43800
15652	0.9000	52.0	2.237474	1.053535	3260.0	2.237474	37.80	-122.41	5.00001
15657	2.5166	52.0	2.839075	1.184049	3436.0	1.621520	37.79	-122.41	2.75000
15659	1.7240	52.0	2.278566	1.082348	4518.0	1.780142	37.79	-122.41	2.25000
15795	2.5755	52.0	3.402576	1.058776	2619.0	2.108696	37.77	-122.42	3.25000
15868	2.8135	52.0	4.584329	1.041169	2987.0	3.966799	37.76	-122.41	2.60300

5.00001
0.14999
PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>

Рисунок 1.11.1 – Результат тестирования программы для задания №11

1.12 Задание №12

Используя метод `apply()`, вывести на экран название признака и его среднее значение.

Реализация программы представлена в Листинге Л.

Листинг Л – Реализация задания №12

```

_ = df.apply(lambda x: print(f"{x.name}: {x.mean()}"))

```

Результат тестирования программы представлен на Рисунке 1.12.1.

The screenshot shows a Jupyter Notebook with a dark theme. At the top, there are tabs for files: 1.6.py, 1.8.py, 1.9.py, and 1.5.py. The active tab is 1.6.py, showing a file explorer path: Практические работы > Практическая работа 1 > 1.6.py > ...

The code in the notebook is as follows:

```

14
15
16 df.info()
17
18
19 print(df.isna().sum())
20
21
22 print(df.loc[(df["HouseAge"] > 50) & (df["Population"] > 2500)])
23
24
25 print(df["Price"].max())
26 print(df["Price"].min())
27
28
29 _ = df.apply(lambda x: print(f"{x.name}: {x.mean()}"))

```

The output of the code is displayed in the terminal pane at the bottom. It shows a table of data with 10 columns and 6 rows of data, followed by a summary of statistics for each column.

15652	0.9000	52.0	2.237474	1.053535	3260.0	2.237474	37.80	-122.41	5.00001
15657	2.5166	52.0	2.839075	1.184049	3436.0	1.621520	37.79	-122.41	2.75000
15659	1.7240	52.0	2.278566	1.082348	4518.0	1.780142	37.79	-122.41	2.25000
15795	2.5755	52.0	3.402576	1.058776	2619.0	2.108696	37.77	-122.42	3.25000
15868	2.8135	52.0	4.584329	1.041169	2987.0	3.966799	37.76	-122.41	2.60300
5.00001	0.14999								

Summary statistics:

```

MedInc: 3.8706710029069766
HouseAge: 28.639486434108527
AveRooms: 5.428999742190376
AveBedrms: 1.096675149606208
Population: 1425.4767441860465
AveOccup: 3.0706551594363742
Latitude: 35.63186143410853
Longitude: -119.56970445736432
Price: 2.068558169089147

```

The terminal prompt is `(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>`.

Рисунок 1.12.1 – Результат тестирования программы для задания №12

1.13 Задание №13

Дан текст на английском языке. Необходимо закодировать его с помощью азбуки Морзе, где каждой букве соответствует последовательность точек и тире. Например, буква «g» превратится в строку «--.». В переменной morze для удобства хранится словарь соответствия латинских букв коду Морзе. morze = {'a': '-.-', 'b': '-...-', 'c': '-.-.-', 'd': '-..-', 'e': '.', 'f': '..-.-', 'g': '--.-', 'h': '....-', 'i': '..-', 'j': '---.-', 'k': '-.-.-', 'l': '-.-..', 'm': '--', 'n': '-.-', 'o': '---', 'p': '---.-', 'q': '---.-', 'r': '-.-.', 's': '...', 't': '-.-', 'u': '..-.-', 'v': '...-.-', 'w': '-.-.-', 'x': '-.-.-', 'y': '-.-.-', 'z': '---.-'}

Реализация программы представлена в Листинге М.

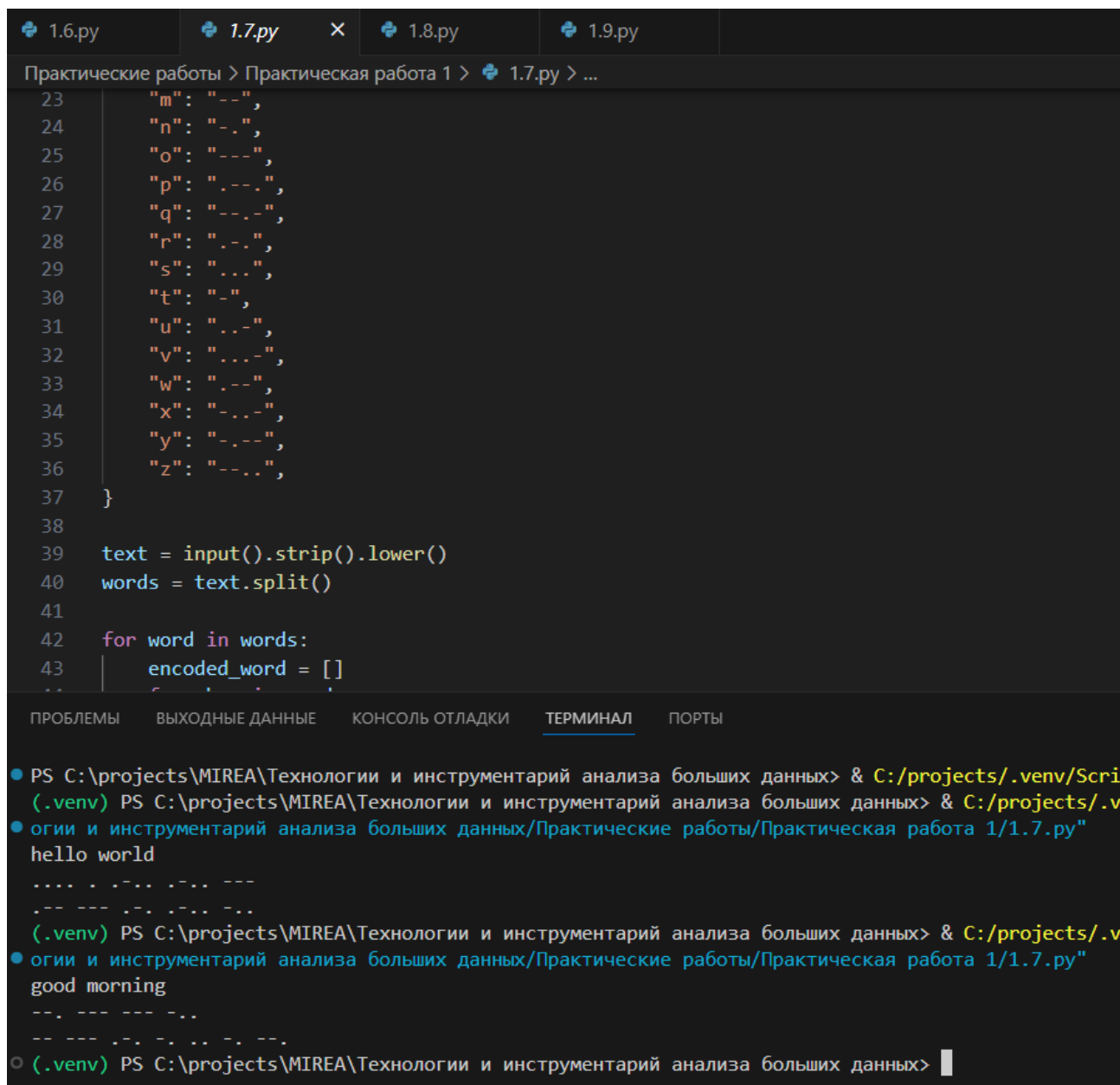
Листинг М – Реализация задания №13

```
morze = {
    "a": ".-",
    "b": "-...",
    "c": "-.-.",
    "d": "-..",
    "e": ".",
    "f": "..-.",
    "g": "--.",
    "h": "....",
    "i": "..",
    "j": ".---",
    "k": "-.-",
    "l": "-...",
    "m": "--",
    "n": "-.",
    "o": "---",
    "p": ".--.",
    "q": "--.-",
    "r": ".-.",
    "s": "...",
    "t": "-",
    "u": "..-",
    "v": "...-",
    "w": ".--",
    "x": "-...-",
    "y": "-.-.",
    "z": "--.."
}

text = input().strip().lower()
words = text.split()

for word in words:
    encoded_word = []
    for char in word:
        if char in morze:
            encoded_word.append(morze[char])
    print(" ".join(encoded_word))
```

Результат тестирования программы представлен на Рисунке 1.13.1.



```
1.6.py 1.7.py x 1.8.py 1.9.py
Практические работы > Практическая работа 1 > 1.7.py > ...
23     "m": "--",
24     "n": "-.",
25     "o": "---",
26     "p": ".--.",
27     "q": "---.",
28     "r": "-.-.",
29     "s": "...",
30     "t": "-",
31     "u": "-.-",
32     "v": "...-",
33     "w": ".--",
34     "x": "-.-.-",
35     "y": "-.-.-",
36     "z": "-.-.-.",
37 }
38
39 text = input().strip().lower()
40 words = text.split()
41
42 for word in words:
43     encoded_word = []
44     for letter in word:
45         if letter in alphabet:
46             shift = alphabet.index(letter) + 1
47             new_letter = alphabet[(shift) % len(alphabet)]
48             encoded_word.append(new_letter)
49         else:
50             encoded_word.append(letter)
51     print(' '.join(encoded_word))
52
53 if __name__ == '__main__':
54     main()
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```
PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python.exe C:/projects/MIREA/Технологии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.7.py
hello world
.... .-.. .-.. ---
..- -.. .-.. -..
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python.exe C:/projects/MIREA/Технологии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.7.py
good morning
--. --- --- -..
-- --- .-.. -.. -..
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных>
```

Рисунок 1.13.1 – Результат тестирования программы для задания №13

1.14 Задание №14

В некотором городе открывается новая служба по доставке электронных писем. Необходимо наладить систему регистрации новых пользователей. Регистрация должна работать следующим образом: если новый пользователь хочет зарегистрироваться на сайте, то он должен послать системе запрос name со своим именем. Система должна определить, существует ли уже такое имя в базе данных. Если такого имени не существует, то оно заносится в базу данных системы и пользователю возвращается ответ "ОК", подтверждающий успешную

регистрацию. А если пользователь с таким именем уже существует, то система должна сформировать новое имя и выдать его пользователю в качестве подсказки, при этом сама подсказка также добавляется в базу данных. Новое имя формируется следующим образом: к name последовательно приписываются числа, начиная с 1 (name1, name2 и так далее), и среди них находят такое наименьшее i, что namei еще не содержится в системе.

Реализация программы представлена в Листинге Н.

Листинг Н – Реализация задания №14

```
import string

data = []
result = []
for _ in range(int(input())):
    name = input()
    if name not in data:
        result.append("OK")
    else:
        i = 1
        while name in data:
            name = name.rstrip(string.digits) + str(i)
            i += 1
        result.append(name)
        data.append(name)
print(*result, sep="\n")
```

Результат тестирования программы представлен на Рисунке 1.14.1.

```
18
19 import string
20
21
22 data = []
23 result = []
24 for _ in range(int(input())):
25     name = input()
26     if name not in data:
27         result.append("OK")
28     else:
29         i = 1
30         while name in data:
31             name = name.rstrip(string.digits) + str(i)
32             i += 1
33         result.append(name)
34     data.append(name)
35 print(*result, sep="\n")
36
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ + ~ 1: Py

PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/Activate.ps1
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> & C:/projects/.venv/Scripts/python.exe C:\projects\MIREA\Технологии и инструментарий анализа больших данных/Практические работы/Практическая работа 1/1.8.py
3
b
b
b
OK
b1
b2
(.venv) PS C:\projects\MIREA\Технологии и инструментарий анализа больших данных> |

Рисунок 1.14.1 – Результат тестирования программы для задания №14

1.15 Задание №15

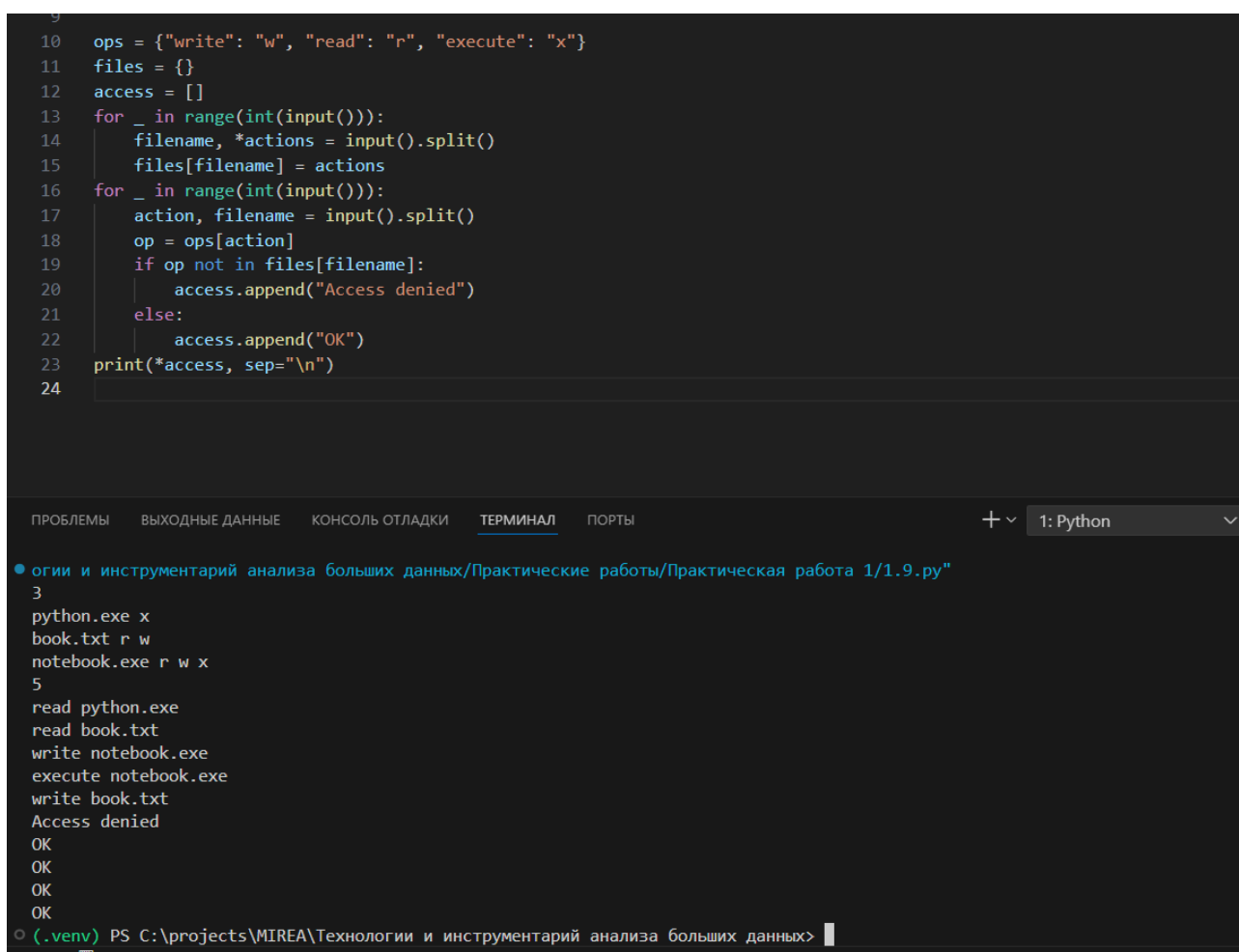
Необходимо создать программу обработки запросов пользователей к файловой системе компьютера. Над каждым файлом можно производить следующие действия: запись – w ("write"), чтение – r ("read"), запуск – x ("execute"). Входные данные На вход программе подаются следующие параметры: число n – количество файлов в файловой системе. В следующих n строках содержится информация с именами файлов и допустимыми действиями (w, x, r), разделенных пробелами. Далее идет число m – количество запросов к файлам вида «операция файл» (обозначение операции: "write", "read", "execute"). Выходные данные Для каждого допустимого запроса программа должна возвращать OK, для недопустимого – Access denied.

Реализация программы представлена в Листинге О.

Листинг О – Реализация задания №15

```
ops = {"write": "w", "read": "r", "execute": "x"}
files = {}
access = []
for _ in range(int(input())):
    filename, *actions = input().split()
    files[filename] = actions
for _ in range(int(input())):
    action, filename = input().split()
    op = ops[action]
    if op not in files[filename]:
        access.append("Access denied")
    else:
        access.append("OK")
print(*access, sep="\n")
```

Результат тестирования программы представлен на Рисунке 1.15.1.



The screenshot displays a Python IDE with a dark theme. The top pane shows the code from Listing O, which is a Python script for file access control. The bottom pane shows the terminal output of the program's execution. The terminal window has tabs for 'ПРОБЛЕМЫ', 'ВЫХОДНЫЕ ДАННЫЕ', 'КОНСОЛЬ ОТЛАДКИ', 'ТЕРМИНАЛ', and 'ПОРТЫ', with 'ТЕРМИНАЛ' being the active tab. The terminal output shows the program's execution with inputs and corresponding outputs.

```
ops = {"write": "w", "read": "r", "execute": "x"}
files = {}
access = []
for _ in range(int(input())):
    filename, *actions = input().split()
    files[filename] = actions
for _ in range(int(input())):
    action, filename = input().split()
    op = ops[action]
    if op not in files[filename]:
        access.append("Access denied")
    else:
        access.append("OK")
print(*access, sep="\n")
```

Terminal Output:

```
python.exe x
book.txt r w
notebook.exe r w x
5
read python.exe
read book.txt
write notebook.exe
execute notebook.exe
write book.txt
Access denied
OK
OK
OK
OK
```

Рисунок 1.15.1 – Результат тестирования программы для задания №15